

Neural Networks II

Introduction

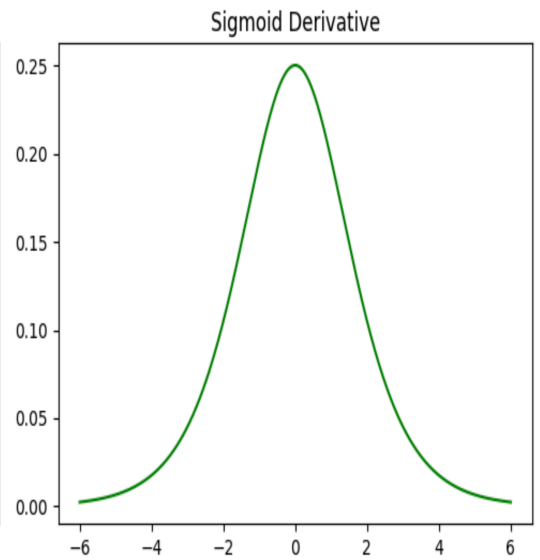
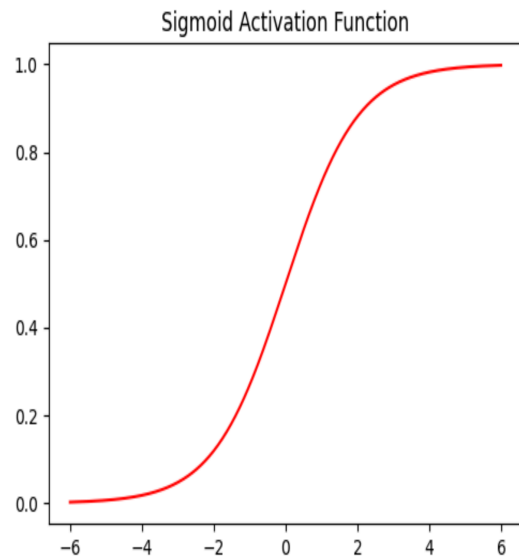
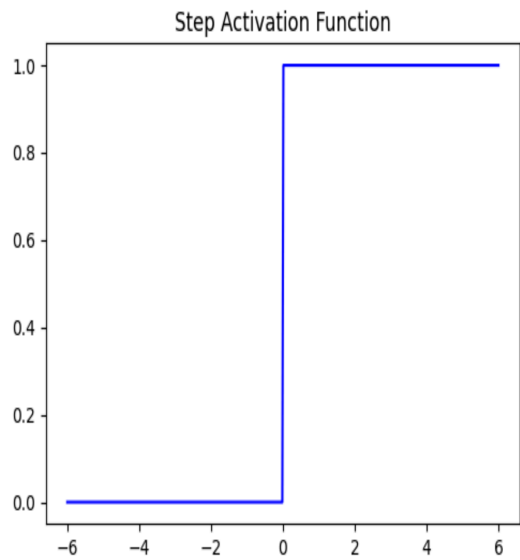
- We continue our work on neural networks.
- This lecture focuses on some key components and challenges.

Activation functions

- McCulloch - Pitts neuron and the Perceptron was motivated by neurons in the brain.
- Utilised a step function to mimic a neuron "firing".
- Not good for gradient descent $\Rightarrow$ replace with a differentiable approximation of the step function.
- The "classic" activation function is the sigmoid activation function:

$$f_s(x) = \frac{1}{1 + \exp(-x)}$$

- Nice derivative: $\frac{df_s(x)}{dx} = f_s(x)(1 - f_s(x))$



Backpropagation for MLP with one neuron in each layer

- Consider MLP with 4 layers and 1 neuron in each layer.
- Let us do Backpropagation with this network.

 No description has been provided for this image

The vanishing gradient problem

- As the number of layers increase, the number of < 1 numbers get multiplied.
- Gradients "vanish", training lower layers becomes difficult.

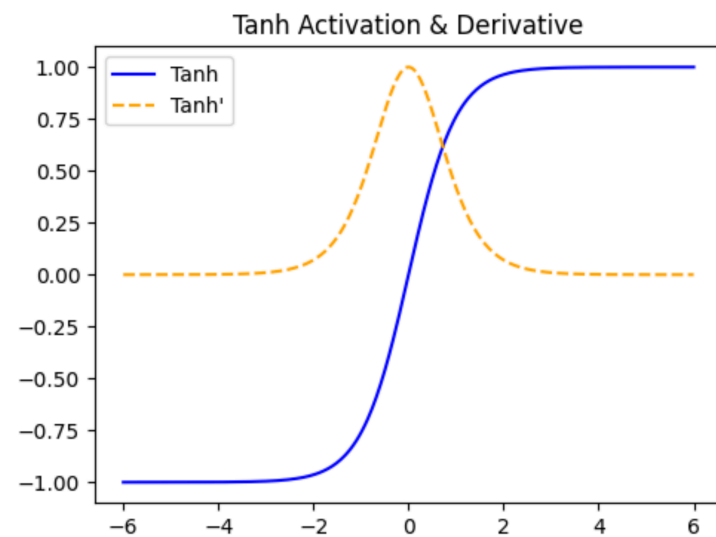
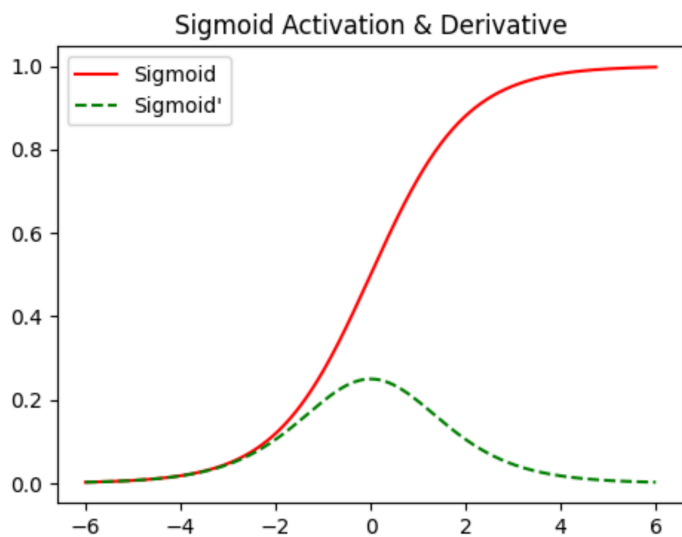
What about other activation functions?

- Another classical choice:

$$f_t(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Also nice derivative:

$$\frac{df_t(x)}{dx} = 1 - f_t(x)^2$$



What about other activation functions?

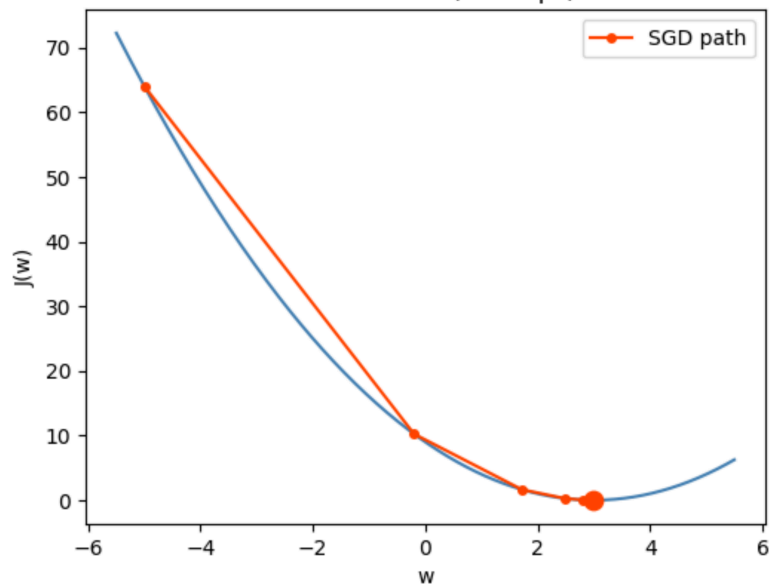
- Tanh has slightly better behavior than sigmoid, but still issues.
- Modern activation functions handle this problem much better (covered in FYS3033).

Gradient descent with momentum

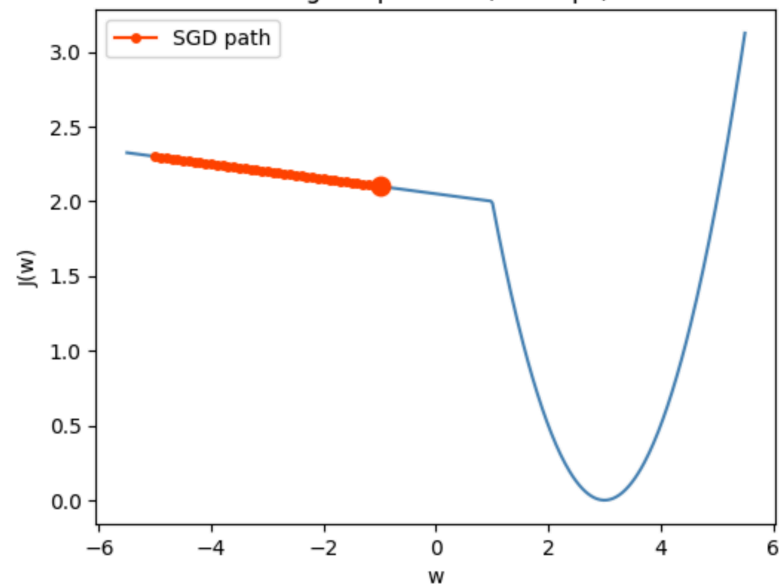
Momentum

- Standard gradient descent can have slow convergence
 - Slow to train! Draw example $\Rightarrow$

Smooth bowl (40 steps)



Long flat plateau (40 steps)



Momentum

- Idea: incorporate information from previous iteration. Keep the "momentum".
- Reminder: $\mathbf{w}_j^l(t+1) = \mathbf{w}_j^l(t) - \gamma \frac{\partial}{\partial \mathbf{w}_j^l} J$
- Now, with momentum: $\Delta \mathbf{w}_j^l = \alpha \mathbf{w}_j^l(t) - \gamma \frac{\partial}{\partial \mathbf{w}_j^l} J$

Momentum for T successive iteration steps

- Momentum effectively increases the learning constant.
- Let $\frac{\partial}{\partial \mathbf{w}_j^l} J = \mathbf{g}(t)$
- $\Delta \mathbf{w}_j^l(T) = -\gamma \sum_{t=0}^{T-1} \alpha^t \mathbf{g}(T-t) + \alpha^T \Delta \mathbf{w}_j^l(0)$
- Now, assume we are in a low curvature point of the loss function $\Rightarrow$ gradient approximately constant!
- $\Delta \mathbf{w}_j^l(T) \approx -\gamma(1 + \alpha + \alpha^2 + \dots + \alpha^{T-1})\mathbf{g}$

Beyond binary classification

- So far, we have focused on binary classification (2 classes).
- Fits well with sigmoid.
- However, we often have more than 2 classes. In these cases, the sigmoid is less suitable.

The softmax function

- The standard choice for multiclass classification is to use a softmax function in the output layer.
- $\hat{y}_k = \frac{\exp(z_k^L)}{\sum_{k'} \exp(z_{k'}^L)}$
- Guarantees that the output lies in the interval $[0, 1]$ and sums to 1.
- Note: **one-hot encoding**

Derivative of the softmax function

- Need to know derivative of softmax function for Backpropagation and gradient descent.

- Need to compute $\frac{\partial}{\partial z_m^L} \hat{y}_k = \frac{\partial}{\partial z_m^L} \frac{\exp(z_k^L)}{\sum_{k'} \exp(z_{k'}^L)}$.
- Key is to realize that we have two cases: $m = k$ and $m \neq k$
- For $m = k$:
$$\frac{\exp(z_k^L) \sum_{k'} \exp(z_{k'}^L) - \exp(z_k^L) \exp(z_m^L)}{(\sum_{k'} \exp(z_{k'}^L))^2}$$
- For $m \neq k$:
$$\frac{-\exp(z_k^L) \exp(z_m^L)}{(\sum_{k'} \exp(z_{k'}^L))^2}$$

Cross-entropy loss

- Can use a squared error like we have done in the past.
- However, using the cross-entropy loss function is much more common:

$$J_{ce} = \sum_{k=1}^{k_L} y_k(i) \log(\hat{y}_k)$$

- Fits nicely with softmax, derivative of cross-entropy loss assuming softmax loss function and one-hot encoded labels.
- Take derivative with respect to preactivation z_m^L . Key idea again, split sum into $m = k$ and $m \neq k$:

$$\frac{\partial}{\partial z_m^L} J_{ce} = -\frac{\partial}{\partial z_m^L} y_k(i) \log(\hat{y}_k) - \frac{\partial}{\partial z_m^L} \sum_{k' \neq m}^{k_L} y_{k'}(i) \log(\hat{y}_{k'})$$

Cross-entropy loss

- Other benefits associated with the cross-entropy loss.
- [See this book for a very nice and detailed explanation.](#)
- Can also be used for binary classification.

Weight initialization

- Need a starting point for our parameters.
- Many heuristics. Different choices for different activation functions. Will focus on sigmoid and tanh.
- Key paper: Glorot and Bengio, Understanding the difficulty of training deep feedforward neural networks, 2010.
- Avoid symmetry!
- Xavier initialization:

$$w_{jm}^l \sim \mathcal{U} \left(-\sqrt{\frac{6}{k_{l+1} + k_{l-1}}}, \sqrt{\frac{6}{k_{l+1} + k_{l-1}}} \right)$$